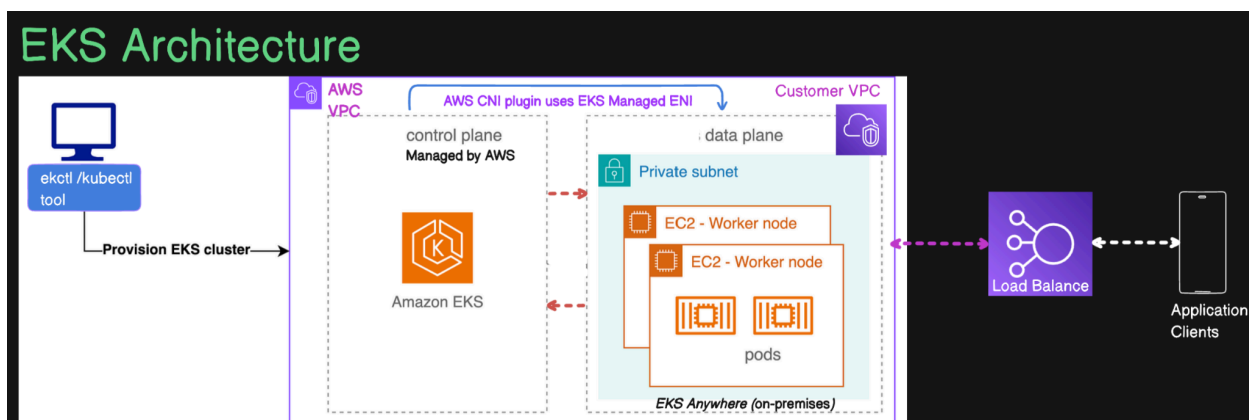
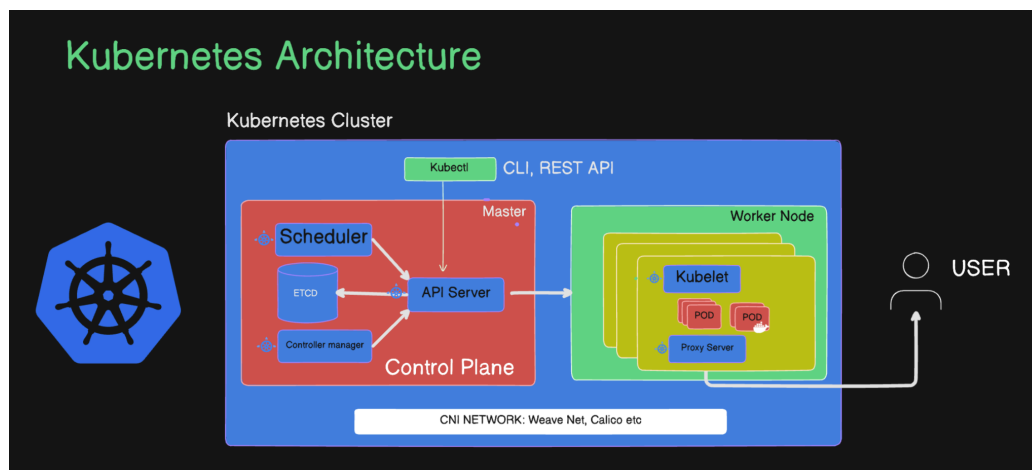
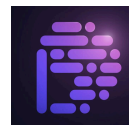


Elastic Kubernetes Service (EKS)

AWS Managed Service: AWS manages the auto-scaling, auto-healing for you. You don't have to do all the hard work, AWS will manage all these services.

AWS Fargate (managed service like AWS Lambda - serverless) is a technology that you can use with Amazon ECS to run containers without having to manage servers or clusters of Amazon EC2 instances. With AWS Fargate, you no longer have to provision, configure, or scale clusters of virtual machines to run containers. This removes the need to choose server types, decide when to scale your clusters, or optimize cluster packing.





Deploy Full Stack Application on AWS EKS

3 simple ways you could create EKS Cluster

1. AWS Console
2. Eksctl
3. Terraform

2. Create Cluster with Eksctl:

Pre-requisites: AWS knowledge

- In a EC2 machine, you will run eksctl command to provide instructions, to trigger eks services
- In the same machine, you will install kubectl and run commands to run pods in cluster
- IAM to give access to the EC2 machine
- AWS CLI to run commands with SSH
- AWS Region: Try to avoid N. Virginia or Ohio

STEPS:

Step-1:

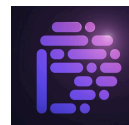
Create an EC2 instance (t2 micro sufficient) because this machine is not for cluster, it is to trigger the ekscluster

Step-2:

Connect the machine in your terminal via SSH

Step-3:

Create an IAM user and attach policy - Administrator (screenshot below)



Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions

Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ Attach policies directly

Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1/1336)



[Create policy](#)

Choose one or more policies to attach to your new user.

Filter by Type			
Search		All types	
< 1 2 3 4 5 6 7 ... 67 >		⚙️	
Policy name	Type	Attached entities	
☐ AccessAnalyzerServiceRolePolicy	AWS managed	0	
☒ AdministratorAccess	AWS managed - job function	1	

Step-4:

You need AWS CLI in the EC2 machine to give IAM user access ([Link](#))

Run this command: to download - awscli2, unzip and install

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip"
-o "awscliv2.zip"
```

```
$ sudo apt install unzip
```

```
$ unzip awscliv2.zip
```

```
$ sudo ./aws/install
```

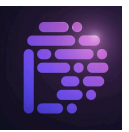
Check AWS version:

```
$ aws --version
```

Step-5: Configure AWS

Give IAM user to this machine (EC2). You need to create Access Key from User

```
$ aws configure
```



Step-6: Install eksctl

```
$ curl --silent --location  
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname  
-s)_amd64.tar.gz" | tar xz -C /tmp
```

```
$ sudo mv /tmp/eksctl /usr/local/bin
```

Check eksctl version: \$ eksctl version

Type: \$ eksctl

Note: eksctl is the tool used to create EKS cluster

Step-6: Install kubectl [Link](#)

Download:

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s  
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

Validate if kubectl downloaded: (You should see OK)

```
$ curl -LO "https://dl.k8s.io/release/$(curl -L -s  
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256"
```

```
$ echo "$(cat kubectl.sha256) kubectl" | sha256sum --check
```

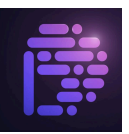
Install kubectl:

```
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Test to ensure the version you installed:

```
$ kubectl version --client
```

Note: kubectl is the tool to interact with kubernetes cluster



Step-6: create eks-cluster

You can play with **t2.micro** or **t2.medium**

```
$ eksctl create cluster --name full-stack-cluster --region us-west-2 --node-type t2.micro  
--nodes-min 2 --nodes-max 2
```

OR,

```
$ eksctl create cluster --name full-stack-cluster --region us-west-2 --node-type t2.medium  
--nodes-min 2 --nodes-max 2
```

Note: This command tells eks to create a VPC and within that the Control Plane will be running. It tells AWS CloudFormation to allocate resources>

AWS CloudFormation is a service that allows you to model, provision, and manage AWS resources using templates, essentially treating infrastructure as code. It automates the deployment of infrastructure, ensuring consistent and repeatable infrastructure setups

Go to: Elastic Kubernetes Services and see all

```
$ kubectl get pods
```

```
$ kubectl get namespace
```

```
$ kubectl get pods -n kube-system
```

```
$ kubectl get nodes
```

Get Cluster:

```
$ eksctl get cluster
```

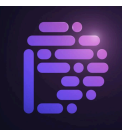
Note: Imagine you have two clusters:

- full-stack-cluster
- nure-cluster

If you want your **kubectl** wants to point full-stack-cluster, your command should be:

```
$ aws eks update-cubeconfig --name full-stack-cluster --region us-west-2
```

```
$ kubectl get nodes (you should see nodes from full-stack-cluster cluster
```



Step-7:

Clone the git repository:

<https://github.com/bongodev/flask-student-attendance-app.git>

Go the eks-manifest directory: `cd eks-manifest`

Step-8:

Create name space or apply namespace.yaml (if you forget namespace)

```
$ kubectl apply -f namespace.yaml
```

Step-9: Run all files from manifest directory to run apps inside pods

```
$ kubectl apply -f namespace.yaml -f mysql-secrets.yml -f mysql-deployment.yml -f mysql-svc.yml -f two-tier-app-deployment.yml -f two-tier-app-svc.yml
```

Or

```
$ kubectl apply -f .
```

Delete all files running inside pod: `kubectl delete -f .`

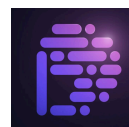
Step-10: Get All PODS

Get all pods: `$ kubectl get all -n two-tier-ns`

Debug Pod:

```
kubectl describe <pod_name (i.e. pod/mysql-6d576469c4-67929)> -n two-tier-ns
```

If you get error like this: Warning FailedScheduling 2m18s (x7 over 32m) default-scheduler 0/2 nodes are available: 2 Too many pods. preemption: 0/2 nodes are available: 2 No preemption victims found for incoming pod.



Check pod log:

```
kubectl logs mysql-98596f974-5crh5 -n two-tier-ns
```

Delete Cluster: `eksctl delete cluster --name <cluster_name> --region us-west-2`

Delete Cluster

```
$ eksctl delete cluster --name <cluster_name> --region <region_name(us-west-2)>
```

Important Notes:

- If you created any other AWS resources **manually** (like RDS, Load Balancers, S3 buckets, IAM roles, etc.), those **will not be deleted**. You'll have to delete them manually.
- If you used a **custom VPC**, `eksctl` won't delete that either.

Optional Manual Cleanup:

If you want to be super sure everything is removed, check these:

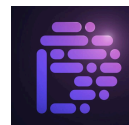
1. CloudFormation Stacks

`eksctl` uses CloudFormation under the hood. Go to the **CloudFormation Console**, and delete any remaining stacks related to your cluster.

2. VPCs

Check your **VPC dashboard** to make sure the VPC created for the cluster is gone (if it was created by `eksctl`).

3. IAM Roles



Delete any cluster-specific IAM roles (optional, and only if you're sure nothing else uses them).

4. EBS Volumes & Load Balancers

Check for leftover **EBS volumes** and **ELBs** in the EC2 dashboard (they might not be auto-deleted in edge cases).